

Database Schema Documentation

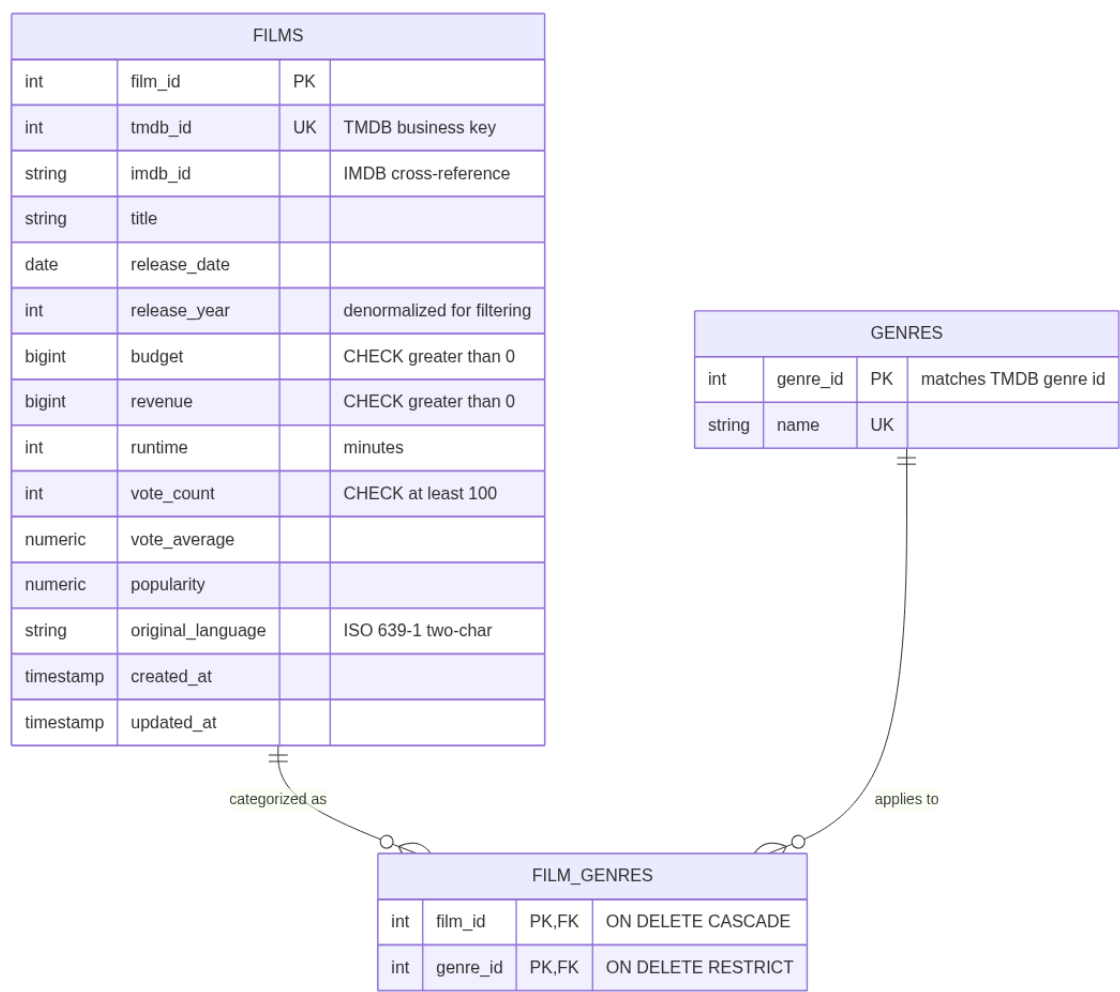
Box Office vs. Ratings

This database stores film financial data (budget, revenue, profit) and audience-rating data sourced from the TMDb API, loaded into a local PostgreSQL 17 instance. The schema supports the project's analytical questions: whether films that earn more also rate higher, and whether certain genres show consistent gaps between revenue and rating.

The schema is normalized to approximately Third Normal Form (3NF) by:

- separating films, genres, and the film-genre relationship into their own tables
- avoiding repeated data – a genre name is stored exactly once in `genres` rather than on every film row
- using a bridge table (`film_genres`) to resolve the many-to-many relationship between films and genres

ER Diagram



ER Diagram

Database Overview

The database contains three tables and one view:

- `films`
- `genres`
- `film_genres`
- `v_films_enriched` (view)

Entity Relationship Summary

Table	Purpose
<code>films</code>	Stores one row per movie with its financial, runtime, and rating attributes
<code>genres</code>	Stores TMDB’s standardized genre lookup (~19 rows)

Table	Purpose
film_genres	Bridge table connecting films to genres
v_films_enriched	Denormalized view that joins the three tables and exposes derived profit , roi , and a comma-joined genre list for downstream analytics

Table Documentation

1. films

Purpose

Stores one row per movie that survives the cleaning rules (`vote_count >= 100` , `budget > 0` , `revenue > 0`). This is the central fact-style table – every financial or rating column referenced by the analytics dashboard lives here.

Examples

- Inception (2010) – \$160M budget, \$839M revenue, 8.4 rating
- Cats (2019) – \$95M budget, \$77M revenue, 4.1 rating

Primary Key

- `film_id` – a surrogate auto-incrementing integer. I chose a surrogate over TMDb’s own id so the bridge table is decoupled from the upstream source if I ever need to re-key.

Unique Keys

- `tmdb_id` – TMDb’s stable identifier for the movie. It is the upsert dedup key used by the load script’s `ON CONFLICT (tmdb_id)` clause, which is what makes re-running the script idempotent.

Relationships

- One film can be associated with many genres through `film_genres` .
- Referenced by `film_genres.film_id` .

Constraints

The cleaning rules are enforced as `CHECK` constraints in the table itself, so the database rejects invalid rows even if the Python filter is bypassed:

- `chk_budget_positive` – `budget > 0`
- `chk_revenue_positive` – `revenue > 0`
- `chk_vote_count` – `vote_count >= 100`

Table Structure

Column Name	Data Type	Key	Description
film_id	SERIAL	Primary Key	Surrogate primary key
tmdb_id	INTEGER	Unique	TMDB's stable movie id; drives upsert dedup
imdb_id	TEXT		IMDB cross-reference (e.g. tt1375666) for dashboard deep-links
title	TEXT		English-locale title from TMDB
release_date	DATE		Theatrical release date
release_year	INTEGER		Year extracted from release_date ; stored separately and indexed for filter performance
budget	BIGINT		Production budget in USD (CHECK greater than 0)
revenue	BIGINT		Worldwide box-office revenue in USD (CHECK greater than 0)
runtime	INTEGER		Runtime in minutes
vote_count	INTEGER		Number of TMDB user ratings (CHECK at least 100)
vote_average	NUMERIC(4,2)		TMDB user rating, 0.00~10.00
popularity	NUMERIC(10,3)		TMDB rolling popularity score
original_language	CHAR(2)		ISO 639-1 language code
created_at	TIMESTAMPTZ		Row insert timestamp (DEFAULT NOW())
updated_at	TIMESTAMPTZ		Updated to NOW() on every upsert

2. genres

Purpose

Stores TMDB's standardized genre lookup. The table is loaded once from TMDB's `/genre/movie/list` endpoint and updated only when TMDB introduces or renames a genre.

Examples

- `genre_id = 28` → Action
- `genre_id = 18` → Drama
- `genre_id = 35` → Comedy

Primary Key

- `genre_id` → TMDB's own genre id reused as the primary key. Reusing the upstream id means joins remain valid across re-extracts without a remapping step.

Relationships

- One genre can apply to many films through `film_genres`.
- Referenced by `film_genres.genre_id`.

Table Structure

Column Name	Data Type	Key	Description
<code>genre_id</code>	INTEGER	Primary Key	TMDB's genre id, reused as the primary key
<code>name</code>	TEXT	Unique	Human-readable genre name

3. film_genres

Purpose

Bridge table that resolves the many-to-many relationship between films and genres. Every (film, genre) pairing produces exactly one row.

Example

Inception (`film_id = 27205`) is classified as Action (`genre_id = 28`), Adventure (`genre_id = 12`), and Science Fiction (`genre_id = 878`), so `film_genres` contains three rows for it: (`27205` , `28`) , (`27205` , `12`) , and (`27205` , `878`) .

Composite Primary Key

- (`film_id` , `genre_id`) → using the pair as the primary key prevents the same (film, genre) combination from being inserted twice.

Foreign Keys

- `film_id` → `films.film_id` (ON DELETE CASCADE → deleting a film removes its genre links)
- `genre_id` → `genres.genre_id` (ON DELETE RESTRICT → a genre cannot be deleted while films still reference it)

Relationships

- Implements the many-to-many relationship between films and genres.

Table Structure

Column Name	Data Type	Key	Description
<code>film_id</code>	INTEGER	Composite PK / FK	Film identifier (references <code>films.film_id</code>)
<code>genre_id</code>	INTEGER	Composite PK / FK	Genre identifier (references <code>genres.genre_id</code>)

4. `v_films_enriched` (view)

Purpose

A declarative (non-materialized) view that joins the three tables and exposes derived columns for direct consumption by the Dash analytics application (`app.py`) and pandas. Because the view is declarative, it always reflects the current state of `films` and `film_genres`.

Derived Columns

- `profit` = `revenue` - `budget`
- `roi` = `CASE WHEN budget > 0 THEN revenue::numeric / budget END` (NULL-safe → returns NULL when budget is zero rather than dividing)
- `genres` = `STRING_AGG(g.name, ', ')` → comma-separated list of every genre attached to the film

All passthrough columns from `films` are included, so the view returns one wide row per movie with no JOIN required at query time.

Cardinality Relationships

Parent Table	Child Table	Relationship Type
<code>films</code>	<code>film_genres</code>	One-to-Many
<code>genres</code>	<code>film_genres</code>	One-to-Many
<code>films</code> → <code>genres</code>	through <code>film_genres</code>	Many-to-Many

Normalization Notes (3NF)

The schema satisfies the three lower normal forms:

- **1NF.** TMDb returns the genre list for each film as an array. Splitting that array into individual rows in `film_genres` makes every cell atomic.
- **2NF.** Every non-key column in `films` depends on the whole key (`film_id`). `film_genres` is all-key and therefore 2NF by definition.
- **3NF.** Genre names are stored only in `genres` . They are not duplicated on `films` or `film_genres` , so there is no transitive dependency through a non-key attribute.

Two design choices intentionally trade strict normalization for query performance and analytical clarity:

- `release_year` is stored on `films` even though it is derivable from `release_date` . Because every dashboard filter operates on year, storing and indexing the year directly avoids a per-row `EXTRACT(YEAR FROM ...)` on each query.
- `profit` and `roi` are NOT stored on `films` . They are functionally dependent on `budget` and `revenue` and storing them would violate 3NF and require maintenance on every update. They live in `v_films_enriched` instead.

Data Source

Film data is sourced from **The Movie Database (TMDb) API** through a two-stage extract, because the required fields are split across two endpoints:

1. `/discover/movie` , paginated by `primary_release_year` from 2000 through 2026 with `vote_count.gte=100` , returns the candidate list (id, title, vote stats, popularity, language, release date, genre ids).
2. `/movie/{id}` is called for each candidate to retrieve the financial fields not exposed by `/discover` (`budget` , `revenue` , `runtime` , `imdb_id`).

After the second stage, any film with `budget = 0` or `revenue = 0` is dropped. TMDb reports zero for these fields on many older or less-documented titles, and including them would distort the analysis. The cleaning rules (`vote_count >= 100` , `budget > 0` , `revenue > 0`) are applied in both the Python filter and as `CHECK` constraints on `films` .

Stricter data-quality filter (transform layer). The `> 0` rules above are necessary but not sufficient, because TMDb has two systematic problems for a *box-office* analysis: (1) **streaming-first films** (Netflix/Amazon “*The Gray Man* \$200M+” \$0.45M, *The Irishman*, *Red Notice*) record only a token qualifying-theatrical figure, so they look like nine-figure flops; (2) **placeholder budgets/revenues** of \$1 “\$5 pass `> 0` but produce absurd ROI. `clean_films()` therefore additionally keeps only films with **budget ≥ \$1,000, revenue ≥ \$1,000, and revenue ≥ 5% of budget**, dropping **349 rows**. The base-table `CHECK` constraints intentionally stay loose (`> 0`) so the raw load remains auditable; the stricter rules live in the Python transform and the `films_enriched.csv` export.

Caveat on 2026 data. 2026 is partial year-to-date “ only 36 films loaded so far, and revenue figures continue to climb for recently released titles. Dashboards should either filter to `release_year <= 2025` or label 2026 explicitly as “YTD (incomplete).”

At the time of submission, the loaded database contains **5,659 films, 19 genres, and 14,914 film-genre links** (6,008 films are extracted from TMDb; 349 are dropped by the stricter data-quality filter described above).

Example Relationship Flow

Inception is a film with three genres in TMDb. It is represented across the three tables as follows:

```
films.film_id          = 27205          (Inception, 2010)

film_genres.film_id    = 27205
film_genres.genre_id   = 28
genres.genre_id        = 28
genres.name            = "Action"

film_genres.film_id    = 27205
film_genres.genre_id   = 12
genres.genre_id        = 12
genres.name            = "Adventure"

film_genres.film_id    = 27205
film_genres.genre_id   = 878
genres.genre_id        = 878
genres.name            = "Science Fiction"
```

A single row in `films` joined to three rows in `film_genres` resolves to “Inception is classified as Action, Adventure, and Science Fiction.” The genre name “Action” is stored exactly once in `genres` regardless of how many films are tagged with it – the central benefit of normalization.